

ハッシュ表 (Hash Table)

動的集合の管理: **辞書操作**が必要

$\left\{ \begin{array}{l} INSERT : \text{要素の追加・代入} \\ DELETE : \text{要素の削除} \\ SEARCH : \text{key値を与え、この値を持つ要素を探索} \end{array} \right.$

実現例: **直接アドレス表 (Direct Address Table)**

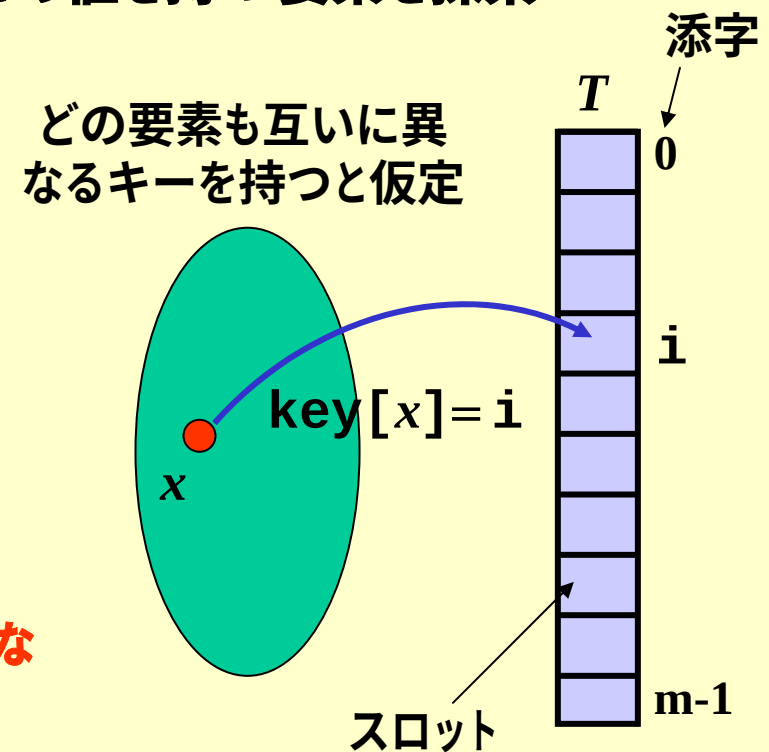
配列 $T[0..m-1]$: キー値を添字とする配列

$$T[i] = \begin{cases} x : \text{key}[x]=i \text{ となる要素が存在} \\ NIL : \text{それ以外} \end{cases}$$

➡ INSERT, DELETE, SEARCHの実行時間 $O(1)$

欠点

キーの取りうる値が大であるとき、非常に大きな記憶場所が必要になる



ハッシュ表

直接アドレス表の欠点:

キーの取りうる値が大であるとき、非常に大きな記憶場所が必要になる

↓ 記憶領域を削減したい

アイデア: “**配列サイズ** $\ll$ **キーの取り得る値の集合のサイズ**” となるような配列を用いる

配列要素 (スロット) の指定: **ハッシュ関数** h を用いる

キー値 k $\longrightarrow$ 配列 $T[h(k)]$ に値を格納する

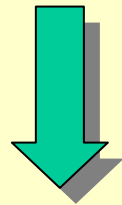
m : 配列 T のサイズ

U : キーの取り得る値の集合

ハッシュ関数 $h: U \rightarrow \{0, 1, \dots, m-1\}$

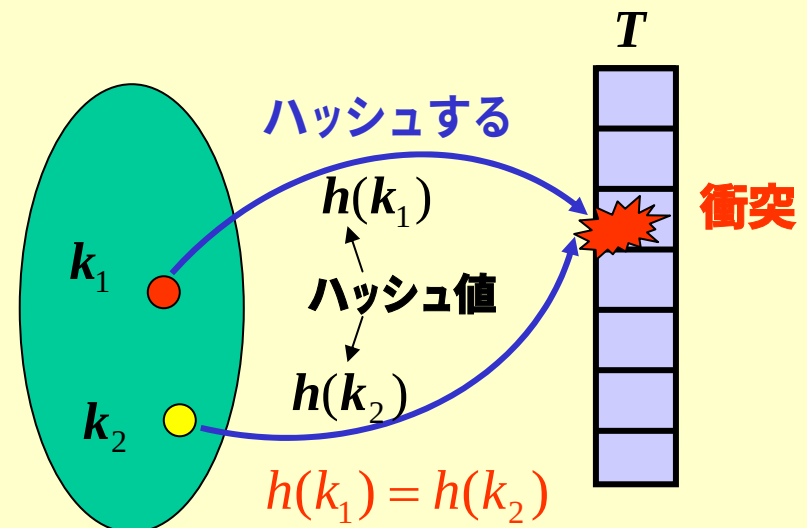
ハッシュ表の問題点

$|U| > m$



異なるキーが同じ
スロットに割り当て
られる可能性あり

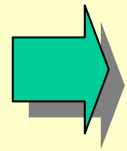
衝突 (collision) が起こる



チェイン法 (Chaining)

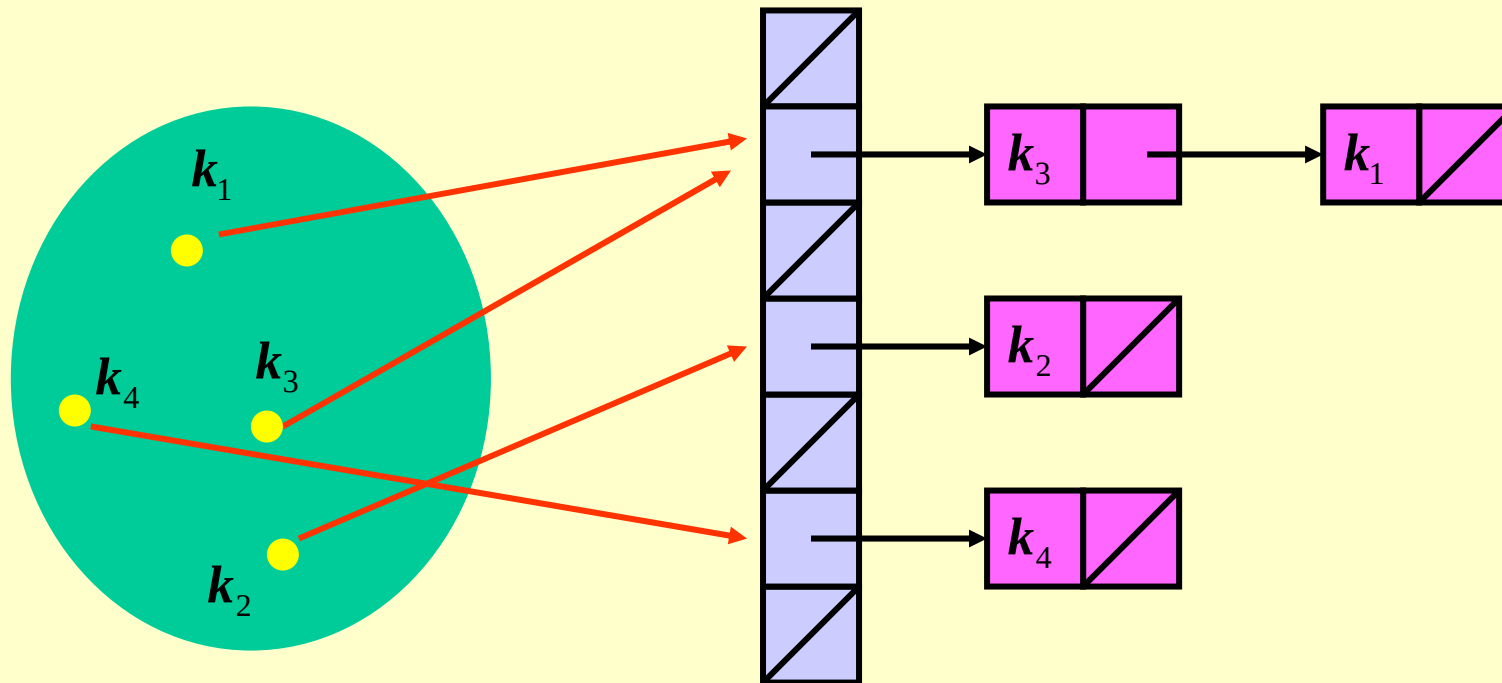
collisionの解決方法 → 連結リストを用いる

- 同じスロットにハッシュされた要素を連結リストに置く
- 各スロットには連結リストの先頭を指すポインタを格納

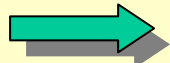


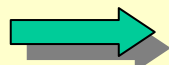
実行時間

- INSERT: リストの先頭に要素を挿入 $O(1)$
- DELETE: 双方向リストなら $O(1)$, 一方向なら SEARCH と同じ
- SEARCH: ???



チェーン法の探索実行時間の解析

- 最悪時: 1つのスロットに全てのデータが存在  $\Theta(n)$
- 任意のキー値について、どのスロットに対してもそこにハッシュされる確率は等しいと仮定 (単純一様ハッシュ)

 各スロットのchain長 (リストの長さ) の期待値 $\sum_{i=1}^n \frac{1}{m} = \frac{n}{m} = \alpha$

要素数 $\nearrow$
スロット数 $\nwarrow$

占有率 (load factor) $\alpha = \frac{n}{m}$

- 失敗に終わる探索 (探索キーを持つ要素がない場合) の平均実行時間
= ハッシュ関数の計算 + chainの平均長 = $\Theta(1 + \alpha)$
高々定数時間 $O(1)$
- 成功する探索 (探索キーを持つ要素が見つかる場合) の平均実行時間
 $\Theta(1 + \alpha)$

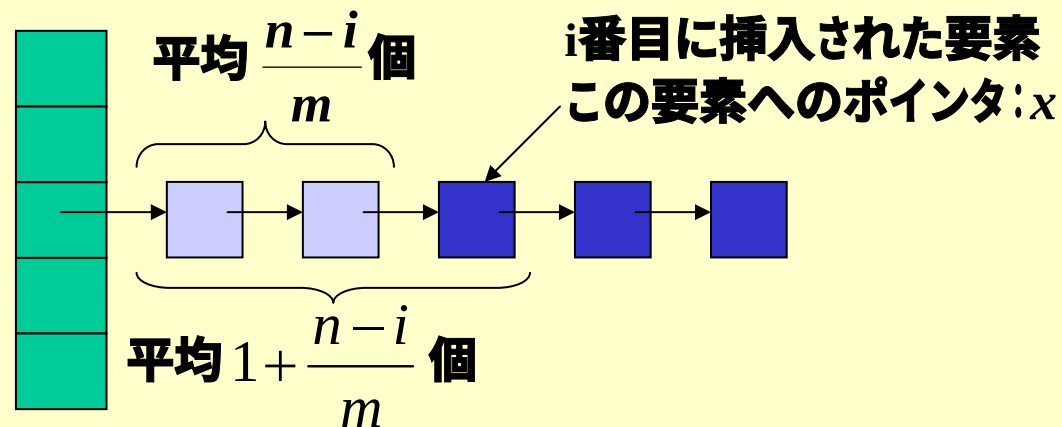
チェイン法における成功する探索平均実行時間の解析詳細

● データ中で*i*番目に挿入された要素 x が探索対象であったとする

- 新しい要素はリストの先頭から挿入
- 探索はリストの先頭から行う
- x の挿入後に挿入される残りのデータ総数は $n-i$ 個

➡ x の挿入後で x が属するリストに挿入されるデータ平均個数 = $\frac{n-i}{m}$

● x を見つけるまでの平均探索時間 = リスト先頭から x までの要素数 = $1 + \frac{n-i}{m}$



全データに対する平均探索時間

→ 平均探索時間 = $\frac{1}{n} \sum_{i=1}^n \left(1 + \frac{n-i}{m}\right) = 1 + \frac{\alpha}{2} - \frac{1}{2m}$

全実行時間 (ハッシュ関数の計算分を加算) = $2 + \frac{\alpha}{2} - \frac{1}{2m} = \Theta(1 + \alpha)$

スロット数 m が要素数 n に比例するなら、 $n = O(m)$

↓

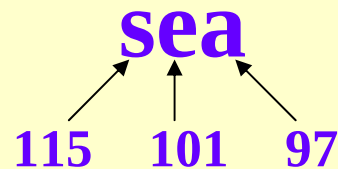
$$\alpha = \frac{n}{m} = \frac{O(m)}{m} = O(1)$$

↓

ハッシュ表の平均探索時間は定数時間となる

ハッシュ関数

- 良いハッシュ関数とは
 - おおよそ単純一様ハッシュの仮定を満たす
- キーの表現
 - キーを自然数として表現して扱う
 - 例: 文字列 sea の自然数表現
 - 1文字は0-127の数値で表現可能(ASCIIコード)
 - 基数128の数字と考えられる



$$115 \times 128^2 + 101 \times 128^1 + 97 \times 128^0 \\ = 1897185$$

ハッシュ関数: 除算法

↓ 剰余演算

$$h(k) = k \bmod m$$

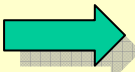
- $m = 2^p$ や $m = 10^q$ などは避ける (用いられないビット、桁などが出る)
- m は素数でかつ $2^p, 10^q$ に近くない数が良い

例: キー $k = 5000$ 、 $m = 359$ とすると

↑
2、10のべき乗に近くない素数を選ぶ

$$h(5000) = 5000 \bmod 359 = 333$$

ハッシュ関数: 乗算法

定数 $0 < A < 1$  $A \approx (\sqrt{5} - 1) / 2$
が良いことがわかっている

$$h(k) = \lfloor m(kA \bmod 1) \rfloor$$

$$= \lfloor m(kA - \lfloor kA \rfloor) \rfloor$$

kA の小数部分を求めている

m の値は重要でない

例: キー $k = 5634$ 、 $m = 256$ 、 $A = 0.618$ とすると

$$\begin{aligned} h(5634) &= \lfloor 256 \times (5634 \times 0.618 - \lfloor 5634 \times 0.618 \rfloor) \rfloor \\ &= \lfloor 256 \times (3481.812 - 3481) \rfloor = \lfloor 207.872 \rfloor = 207 \end{aligned}$$

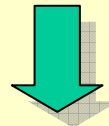
オープンアドレス指定法

ハッシュ表で使われていない領域を有効利用したい

→ 全ての要素をハッシュ表内に格納する

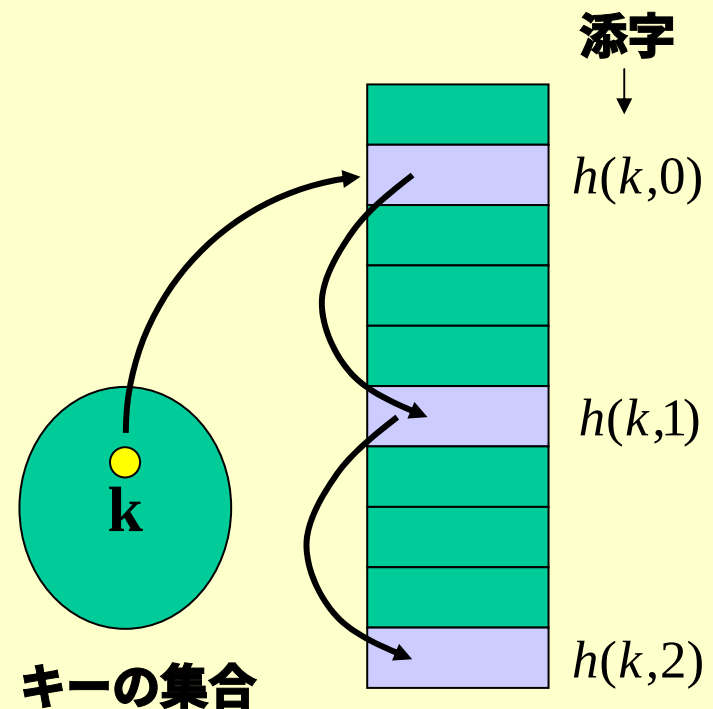
- キー k に対する探査列

$\langle h(k,0), h(k,1), \dots, h(k, m-1) \rangle$



$\langle 0, 1, \dots, m-1 \rangle$

の順列である必要あり



探査列の生成法 $(i=0,1,\dots,m-1)$

1. 線形探査

$$h(k,i) = (h'(k) + i) \bmod m$$

2. 2次関数探査

$$h(k,i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$$

3. ダブルハッシュ法

$$h(k,i) = (h_1(k) + i h_2(k)) \bmod m$$

3つの中では最も多くの探査列を生成することができるため、最も良い性能を示す。